CS699 Data Mining
Complete Project Overview

Classification of Disability Status in U.S. Households

Ho Man Cheng & Weitao Peng
Boston University Metropolitan College | Spring 2026

Section 1 — What Is This Project About?

The goal is to **predict whether a U.S. household contains at least one person with a disability** — a Yes or No answer. This is called a **binary classification problem**.

The data comes from the **2023 American Housing Survey (AHS)**, a large national survey conducted by the U.S. Census Bureau. The survey collects information about households — things like income, housing type, household size, age of members, and health-related questions.

Why does this matter? Knowing which households are likely to have a disabled member helps government agencies and policymakers:

- Allocate accessible housing and financial support
- Plan disability services and healthcare resources
- Identify underserved communities

Section 2 — The Raw Dataset

The original AHS file had:

- ~12,000+ rows (each row = one household)
- 145 columns (each column = one survey question or feature)
- All values are integer codes (e.g., 1 = Yes, 2 = No, -6 = Not applicable)
- Missing values recorded as NA
- One target column called CLASS — value is Yes or No

The key problem from the start: the classes were imbalanced.

- ~30.4% of households = Yes (have a disabled member)
- ~69.6% of households = No

This is a problem because a model could score **70% accuracy by always predicting No** — and still be useless for actually finding disabled households. So we needed special strategies to handle this.

Section 3 — Data Preprocessing — 7 Steps

Raw data is messy. Before training any model, we had to clean and prepare it. Here is what we did and why:

Step 3.1 — Remove rows with missing CLASS label

What: Drop any household row where the disability label (CLASS) is NA.

Why: A model cannot learn from rows where the answer is unknown. These rows are useless for training or testing.

Result: Went from ~12,000+ rows to 12,045 usable rows.

Step 3.2 — Drop the CONTROL column

What: Remove the CONTROL column entirely.

Why: CONTROL is a unique household ID number. It has zero relationship to disability status. If we kept it, the model might memorize IDs instead of learning real patterns.

Step 3.3 — Convert everything to numeric

What: Convert all remaining columns from factor/character type to numbers.

Why: Most R machine learning algorithms require numeric inputs. The AHS already uses integer codes, so this ensured R treated them as numbers, not categories.

Step 3.4 — Median Imputation (fill in missing values)

What: For every column that still has NA values, replace each NA with the median of that column.

Why: We cannot leave gaps in the data — most algorithms crash on NA values. We use the median (not the mean) because the median is more robust to extreme outliers.

Step 3.5 — Near-Zero Variance (NZV) Filter

What: Remove columns that are nearly constant — columns where almost every row has the same value.

Why: A column that is the same for 99.9% of households teaches the model nothing.

Critical decision: We used a lenient threshold (99:1 frequency ratio) instead of the default strict threshold (95:5). Why? Because disability-related features like HHSEE (vision difficulty), HHMEMRY (memory difficulty), and HHCARE (self-care difficulty) are rare, and the default threshold would have deleted the most important predictors.

Result: Went from 144 features down to 118 features.

Step 3.6 — Winsorization (clip extreme outliers)

What: For continuous numerical features (columns with more than 10 unique values), cap values below the 1st percentile up to the 1st percentile, and cap values above the 99th percentile down to the 99th percentile.

Why: Extreme outliers can distort model training — especially for algorithms sensitive to scale (SVM, KNN, Neural Networks). We only winsorized continuous features (10+ unique values); binary features were deliberately skipped to avoid collapsing their values to zero.

Step 3.7 — Center and Scale (Standardize)

What: Transform every feature to have mean = 0 and standard deviation = 1.

Why: Algorithms like SVM, KNN, and Neural Networks are extremely sensitive to the scale of input variables. A feature measured in thousands would otherwise dominate a feature measured in ones. Centering and scaling puts all features on the same playing field.

After preprocessing:

- 9,636 rows for training
- 2,409 rows for testing (held out completely, never touched during training)
- 118 features ready for model training

Section 4 — Class Balancing — 4 Methods

Even after preprocessing, the training set was still ~30% Yes / ~70% No. We applied four different techniques to balance the classes before training and tested all four to see which produced the best models.

Critical rule: Balancing was applied **only to the training set**. The test set was kept in its original imbalanced state — because that reflects real-world conditions.

Method 1 — Random Undersampling

What: Randomly remove No-class rows until Yes and No are equal in count.

Why: Simple, fast, and forces the model to focus equally on both classes. The downside is we discard a lot of majority-class data (~5,800 rows kept instead of 9,636).

Method 2 — Random Oversampling

What: Randomly duplicate Yes-class rows until Yes and No are equal.

Why: Keeps all original data. The downside is the model sees the same minority examples many times, which can lead to overfitting.

Method 3 — SMOTE (K=5)

What: Synthesize new minority rows by interpolating between a real Yes row and its 5 nearest Yes neighbors.

Why: Creates more diverse minority examples than simple oversampling. Avoids exact duplicates while expanding minority representation.

Method 4 — ROSE

What: Use a smoothed bootstrap approach to generate a completely synthetic balanced dataset from both classes.

Why: More aggressive synthetic generation than SMOTE. In practice on our dataset, ROSE underperformed — its samples blurred the decision boundary.

Section 5 — The 9 Classification Algorithms

We trained 9 different types of models on each balanced dataset. **4 balancing methods × 9 algorithms = 36 models total.** Each model has a number: Model 1 = Under + LR, Model 2 = Under + DT, …, Model 36 = ROSE + XGBoost.

#	Algorithm	Family	Why Included
1	Logistic Regression	Linear	Simple baseline; interpretable; fast
2	Decision Tree	Tree	Interpretable; handles binary features well
3	Random Forest	Ensemble Tree	Strong with noisy data; robust to overfitting
4	GBM	Ensemble Tree	Sequential boosting; usually high AUC
5	XGBoost	Ensemble Tree	State-of-the-art for tabular data
6	SVM (Radial Kernel)	Kernel	Effective in high-dimensional spaces (118 features)
7	KNN	Distance	Non-parametric; captures local structure
8	Naive Bayes	Probabilistic	Fast; works well with noisy features
9	Neural Network	Neural	Captures nonlinear interactions; single hidden layer

Section 6 — Model Training & Tuning

Each of the 36 models was trained using **5-fold cross-validation** — but only for **hyperparameter tuning**, not for final evaluation.

What is 5-fold CV? Split the training data into 5 equal chunks. Train on 4 chunks, validate on 1. Repeat 5 times rotating which chunk is the validation set. Average the results to pick the best hyperparameters (e.g., tree depth, number of trees, regularization strength).

Why CV only for tuning, not for evaluation? Because final scores must come from the held-out test set — data the model has never seen. Using CV for final scoring would artificially inflate the numbers.

One special case — XGBoost: The xgboost package in R 3.x is incompatible with the caret framework. So we wrote a **manual 5-fold CV loop from scratch** just for XGBoost, while all other 8 algorithms used caret's built-in tuning.

Section 7 — Evaluation Protocol

After training, each model made predictions on the **2,409 unbalanced test rows** using a **fixed decision threshold of 0.5**.

Why a fixed threshold? So every model is compared at the same operating point — a fair, apples-to-apples comparison. Allowing per-model threshold tuning would be data leakage (fitting the threshold to test data).

Metrics tracked for each model:

Metric	What It Measures
Yes TPR	Of all actual Yes households, how many did we correctly call Yes? (most important)
No TPR	Of all actual No households, how many did we correctly call No?
Precision	Of all households we called Yes, how many actually were Yes?
F1 Score	Harmonic mean of Precision and Recall
AUC	Overall ranking ability across all possible thresholds
Kappa	Agreement above chance level
MCC	Matthews Correlation Coefficient — robust overall score

Extra-credit targets set by the professor: Yes TPR >= 0.77 AND No TPR >= 0.73 (both simultaneously = +20% extra credit)

Section 8 — Results Across All 36 Models

After running all 36 models, the key patterns were:

- Undersampling consistently produced the **highest Yes TPR** — but sometimes at the cost of No TPR
- Oversampling and SMOTE generally favoured **No TPR** — the majority class
- ROSE performed **worst on Yes TPR** — its synthetic samples blurred binary disability features
- Tree ensembles (RF, GBM, XGBoost) led on **AUC** — strong overall discrimination
- Only a **handful of models** crossed both thresholds simultaneously

Section 9 — The Best Model

Model 2: Random Undersampling + Decision Tree — the **only model** out of 36 to meet both extra-credit thresholds simultaneously.

Metric	Score	Required	Status
Yes TPR	**0.7730**	>= 0.77	**PASSED**
No TPR	**0.7322**	>= 0.73	**PASSED**
AUC	**0.7990**	—	
F1 (Yes)	**0.6140**	—	
Kappa	**0.4582**	—	

Why did a simple Decision Tree win over XGBoost and Random Forest?

- XGBoost had a higher AUC (0.86) — meaning it ranked Yes above No better across all thresholds
- But at the fixed threshold of 0.5, XGBoost's probability outputs were clustered near 0.5, so many true Yes cases fell just below the cutoff
- The Decision Tree, trained on the 50/50 undersampled data, produced **confident, polarized probabilities** — Yes cases got scores like 0.8–0.9, not borderline 0.48
- The Decision Tree's binary splitting rules matched perfectly with binary disability indicator features (HHSEE, HHMEMRY, HHCARE, HHERRND) — these became the top nodes of the tree and created clean, decisive splits

Section 10 — Deliverables Produced

File	Description
`project_code.R`	All 36 model blocks in a single R script (no loops over models)
`preprocessed_data.csv`	Full cleaned dataset — 17.3 MB
`initial_train.csv`	Training split — 12.1 MB
`initial_test.csv`	Test split — 5.2 MB
`Cheng_HoMan_Peng_Weitao_Report.docx`	45–50 page final report with all tables, charts, and analysis
`Cheng_HoMan_Peng_Weitao_Slides.pptx`	12-slide, 10-minute presentation deck
`Presentation_Script.html`	Word-for-word script for both presenters

`QA_Preparation.html`	6 Q&A questions with full answers and fallback one-liners
`Cheng_HoMan_Peng_Weitao.zip`	Final submission archive containing all required files

Section 11 — Key Takeaways

1. Preprocessing quality matters more than algorithm choice

The lenient NZV threshold that preserved disability indicator features had more impact than which algorithm we picked. Getting the data right first is the foundation of everything.

2. Accuracy is a misleading metric under class imbalance

We never used accuracy as our primary metric — Yes TPR and No TPR told the real story. A 70% accurate model that always predicts No is worthless for this task.

3. Simple models can beat complex ones

When the evaluation metric favours clean decisions over probabilistic ranking, a pruned Decision Tree can outperform XGBoost and Random Forest.

4. Balancing method choice is critical

Undersampling doubled the Yes TPR compared to ROSE. The choice of balancing method had a bigger effect than the choice of algorithm in many cases.

5. AUC does not equal task performance

The models with the highest AUC (XGBoost, Random Forest) did not win on the metric the professor actually graded — per-class TPR at a fixed threshold of 0.5.